

输入加速

```
// 禁止混用 cin / scanf 和 cout / print
#define ios ios::sync_with_stdio(false);cin.tie(0);
```

常用宏

```
using ll = long long;
#define ios ios::sync_with_stdio(false);cin.tie(0);
#define mod(t, m) ((t) % (m) + (m)) % (m)
#define range(a) a.begin(), a.end()
```

输出浮点数

double 使用 printf 设定 .nlg 输出 n 位有效数字
cout 则使用 setprecision 设定输出总位数

最大公因数 GCD 欧几里得算法

```
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

最小公倍数 LCM

```
int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}
```

位运算小技巧 存在 int a, n

$a \ll n$ 等价于 $a * 2^n$ ，将 a 的二进制左移 n 位

$a \gg n$ 等价于 $\frac{a}{2^n}$ ，将 a 的二进制右移 n 位

$a \& 1$ 快速判断 a 的奇偶性，真为奇数，假为偶数

判断第 n 位是否为 1：使用 $(a \gg n) \& 1$ 或 $a \& (1 \ll n)$ 。

$a > 0 \&\& (a \& (a - 1)) == 0$ 快速判断 a 是否为 2 的幂（等价于 $a == 2^x$ ）

$\&$ 按位与：两个均为 1 才出 1

$|$ 按位或：有 1 出 1

注：当 a 为偶数时，使用 $a | 1$ 可以实现 $a + 1$

存在公式： $(a \& b) + (a | b) = a + b$

$\wedge$ 异或（XOR）：相同取 0，不同取 1，存在定理 $(a \oplus a = 0, a \oplus 0 = a)$

STL 容器小技巧 假定存在 `vector<int> a`

1. `sort(a.begin(), a.end())` 默认升序排序

基本类型可加默认降序排序器 `greater<ll>` 或自定义 lambda: `[](int a, int b){ return a>b; }`

自定义类型需要重载运算符 (`operator>`) 或自定义 lambda

当且仅当需要保持原顺序时使用 `stable_sort` (其他情况一律 `sort`)

2. 字符串 `String` 在比较和排序时按照字典序进行排序和比较。

3. `reverse(a.begin(), a.end())` 不包括 `a.end()` 指定的元素 (即翻转范围为 `[begin, end)`)。

4. `lower_bound` 和 `upper_bound` 使用前都必须保证容器有序。

5. `priority_queue`、`map` 和 `set` 自动维护升序 (降序需重载或使用自定义函数), 其中 `map` 和 `set` 可以去重。

6. `find()` 查找函数对于普通容器在找不到时返回右侧 (如 `a.end()`), 对于字符串返回 `string::npos`。

7. `a.erase(remove(a.begin(), a.end(), value), a.end())` 删除数组 `a` 中所有值为 `value` 的元素

`a.erase(remove_if(a.begin(), a.end(), fun), a.end())` 删除数组 `a` 中所有条件满足 `fun` 的元素

7. `vector` 的 `pop_back`、`queue` 的 `pop`、`deque` 的 `pop_back` 和 `pop_front` 都是 $O(1)$ 的时间复杂度。

比赛小技巧

1. 转盘时可以固定一个, 然后枚举另外两个

2. 对于回文字符串, 优先考虑枚举长度

3. 寻找 `k` 的倍数构成序列, 做 `k` 余数序列, 相同余数但不同的两个位置, 必然构成 `k` 倍数的连续序列

4. 寻找不重叠的区间, 对右端点进行升序排序寻找即可, 仅无权计数版; 带权总和和最大 $\rightarrow$ 贪心失效, 改 DP+二分

数值表

`n` 规模算法对应表

> 评测机每秒上限 10^8 次

数据规模 n	可过复杂度	典型算法
$\leq 10 - 12$	$O(n!)$	全排列暴力
$\leq 20 - 25$	$O(2^n)$ 或 $O(2^n \cdot n)$	子集枚举、状压
≤ 500	$O(n^3)$	Floyd、区间 DP
≤ 5000	$O(n^2)$	朴素 DP、双重循环
$\leq 10^5$	$O(n \log n)$ 或 $O(n\sqrt{n})$	排序、堆 Dijkstra、线段树
$\leq 10^6$	$O(n)$ 或 $O(n \log n)$	BFS、前缀和、筛
$\leq 10^8/10^9$	$O(\sqrt{n})$ 、 $O(\log n)$ 、 $O(1)$	二分、数学、快速幂

2^i 对应表

i	2^i	10^i 近似值	备注
10	1024	$\approx 10^3$	
16	65536	$\approx 6.5 \times 10^4$	
17	131072	$\approx 1.3 \times 10^5$	
18	262144	$\approx 2.6 \times 10^5$	
19	524288	$\approx 5.2 \times 10^5$	
20	1048576	$\approx 10^6$	
24	16777216	$\approx 1.68 \times 10^7$	
30	1073741824	$\approx 10^9$	
31	2147483648	$\approx 2.15 \times 10^9$	int 溢出线
32	4294967296	$\approx 4.3 \times 10^9$	unsigned 极限
40	$\approx 1.1 \times 10^{12}$	$\approx 1.1 \times 10^{12}$	
60	$\approx 1.15 \times 10^{18}$	$\approx 1.15 \times 10^{18}$	10^{18} 基准线
63	$\approx 9.22 \times 10^{18}$	$\approx 9.22 \times 10^{18}$	long long 溢出线

二分 最大化最小值

// 目标：找到满足 check(x) 为 true 的最大 x

```
int left = 0, right = MAX_VAL;
while (left < right) {
    int mid = left + ((right - left + 1) >> 1); // 向上取整
    if (check(mid)) { // mid 满足条件，尝试找更大的，[mid, right)
        left = mid;
    } else { // mid 不满足，答案一定在左边，[left, mid-1]
        right = mid - 1;
    }
}
return left; // 或 right
```

二分 最小化最大值

// 目标：找到满足 check(x) 为 true 的最小 x

```
int left = 0, right = MAX_VAL;
while (left < right) {
    int mid = left + ((right - left) >> 1); // 向下取整
    if (check(mid)) { // mid 满足条件，尝试找更小的，[left, mid]
        right = mid;
    } else { // mid 不满足，答案一定在右边，[mid+1, right)
        left = mid + 1;
    }
}
return left; // 或 right
```

BFS 和 DFS

```
vector<vector<int>>> image;    // 邻接表
vector<bool> flag;           // 节点记录
// DFS:沿着一个方向走到头
void dfs(int i,vector<int>& res){
    res.push_back(i);        // 保存当前节点
    flag[i]=true;           // 记录已访问
    // 寻找下一个可以走的位置
    for(int& t:image[i])
        if(!flag[t])
            dfs(t,res);
}
// BFS:逐层遍历
void bfs(int i, vector<int>& res){
    queue<int> q;            //BFS 实现关键
    q.push(i);
    flag[i] = true;
    while(!q.empty()){
        int t=q.front();
        res.push_back(t);
        q.pop();
        // 根据当前节点进行扩散寻找值[子节点]
        for (int& v : image[t]) {
            if (!flag[v]) {
                q.push(v);flag[v] = true;
            }
        }
    }
}
```

并查集

```
class UnionFind {
private:
    /* 连通分量[不同集合]个数 */
    int count;           //连通分量个数
    /* 每个节点的上一节点[祖先] */
    vector<int> parent;   //祖先
    /* 以每个节点为根节点的树大小 */
    vector<int> size;     //大小
public:
    UnionFind(int n) : count(n), parent(n + 1), size(n + 1, 1) {
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
        }
    }
    bool merge(int p, int q){
        int rootP = find(p);int rootQ = find(q);
        if (rootP == rootQ) return 0;
        if (size[rootP] > size[rootQ]) {
            parent[rootQ] = rootP; size[rootP] += size[rootQ];
        } else {
            parent[rootP] = rootQ; size[rootQ] += size[rootP];
        }
        count--;
        return 1;
    }
    bool connected(int p, int q) { return find(q) == find(p); }
    int find(int x) {
        if (x == parent[x]) return x;
        return parent[x] = find(parent[x]);
    }
    int Count() { return count; }
};
```

一维前缀和

```
vector<int>& nums;           //原数组
int n=nums.size();
vector<int> sum(n+1,0);      //前缀和数组
//构造前缀和
for(int i=0;i<n;i++) sum[i+1]=sum[i]+nums[i];
//获取指定范围的和
auto getSum = [&](int left,int right){ return sum[right+1]-sum[left]; };
```

二维前缀和

```
vector<vector<int>>& mat;           //原数组
int n=mat.size(),m=mat[0].size();
vector<vector<int>> sum(n+1,vector<int>(m+1,0));
/* 二维数组计算前缀和 */
for(int i=1;i<=n;i++) for(int j=1;j<=m;j++) sum[i][j] = mat[i-1][j-1] + sum[i-1][j] + sum[i][j-1] - sum[i-1][j-1];
/* 获取指定矩阵和 */
auto getSum = [&](int r1,int c1,int r2,int c2){
    return sum[r2+1][c2+1] - sum[r2+1][c1] - sum[r1][c2+1] + sum[r1][c1];
};
```

二维数组转换成一维数组的变化及还原

```
vector<vector<int>> nums;           // 目标二维数组
int n=nums.size(),m=nums[0].size(); // 二维数组的行和列
vector<int> res(n*m);              // 转换后的目标一维数组
// 转换过程
for(int i=0;i<n;i++)
    for(int j=0;j<m;j++)
        res[i*m+j] = nums[i][j];
vector<vector<int>> new_nums(n,vector<int>(m)); // 还原后的二维数组
// 还原过程
for(int i=0;i<n*m;i++){
    int row = i / m, col = i % m;
    new_nums[row][col] = res[i];
}
```

三维数组转换成一维数组的变化及还原

```
vector<vector<vector<int>>> nums;           // 目标三维数组
int g=nums.size(),n=nums[0].size(),m=nums[0][0].size(); // 三维数组的数量、行和列
vector<int> res(g*n*m);                   // 转换后的目标一维数组
// 转换过程
for(int k=0;k<g;k++)
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            res[k*n*m+i*m+j] = nums[k][i][j];
// 还原后的三维数组
vector<vector<vector<int>>> new_nums(g,vector<vector<int>>(n,vector<int>(m)));
// 还原过程
for(int i=0;i<g*n*m;i++){
    int grw = i / (n*m), row = (i % (n*m)) / m, col = i % m;
    new_nums[grw][row][col] = res[i];    }
```

差分数组

```
class Difference{
    vector<int> diff;           //差分数组

public:
    /* 差分数组的初始化 */
    Difference(const vector<int>& nums){
        assert(!nums.empty()); //检查数组是否不为空
        diff.resize(nums.size());
        diff[0]=nums[0];
        for(size_t i=1;i<nums.size();i++)
            diff[i] = nums[i] - nums[i-1];
    }

    /* 差分数组的操作[val 可正可负] */
    void increment(int i,int j,int val){
        assert(i >= 0 && j < (int)diff.size() && i <= j);
        diff[i]+=val;
        //确保 j 不是最后一个元素
        //右端点为最后一个元素时不用处理
        if(j+1<(int)diff.size())
            diff[j+1]-=val;
    }

    /* 差分数组的还原 */
    vector<int> result() const {
        vector<int> res(diff.size());
        res[0]=diff[0];
        for(size_t i=1;i<diff.size();i++)
            res[i]=res[i-1]+diff[i];
        return res;
    }
}
```

拓扑排序（kanh 入度法）【有向无环图】 $O(n + m)$

```
vector<vector<int>> map(n+1);    //邻接表
vector<int> in(n+1);            //入度记录[关键]
//vector<int> list;              //拓扑排序得到的顶点序列
// 拓扑排序只能求得其中一种合法的顶点序列（满足边方向约束）
// 本题使用 DP 来求得从源点到汇点的路径数量总和
// 设 dp[i]为从所有源点走到 i 的路径数量
vector<int> dp(n+1,0);
// 建立邻接表
for(int i=0;i<m;i++){
    int u,v;cin>>u>>v;        //{出发点,到达点}
    map[u].push_back(v);       //加入邻接表
    in[v]++;                    //记录到达点的入度
}
// 拓扑排序实现的关键:队列
// 每次拿取的是入度为 0 的节点
// 即队列存取的是入度为 0 的节点
queue<int> zero;
// 寻找所有入度为 0 的点,给寻找序列建立基础
for(int i=1;i<=n;i++){
    if(in[i]==0){
        zero.push(i);
        // 该点入度为 0,即代表该点必有一条最大链
        dp[i]=1;
    }
}
while(!zero.empty()){
    // 先拿取一个入度为 0 的节点
    int u = zero.front();
    // 然后删除该节点
    zero.pop();
    // 将该节点加入序列
    //list.push_back(u);
    // 然后更新入度记录
    // 即当前由该点所连的边得到的入度全部清除
    for(int& t:map[u]){
        // 更新入度
        in[t]--;
        // 更新经由该点的路径数量
        // u 节点当前为零入度点,代表必有一条路径经由
        // 故经由该点下一步可达的点的路径中必含有经由 u 节点的路径
        // 则下一点的路径数量必是两点路径数量之和
        dp[t] = (dp[u] + dp[t]) % MOD;
        // 如果当前点入度为 0,就要将其加入队列
        if(in[t]==0) zero.push(t);
    }
}
// 所得到的 list 就是一种拓扑排序序列
int ans = 0;    // 总路径数量是所有汇点（出度为 0）的路径数量之和
// 路径末点必是出度为 0 的节点
for(int i=1;i<=n;i++)
    if(map[i].empty())    ans = (dp[i] + ans) % MOD;
cout<<ans;
```


最小生成树（Prim 算法） $O((n + m)\log n)$

```
const ll INF = 1LL << 60;
struct state {
    int u; ll w;
    bool operator<(const state& a) const {
        return w > a.w;
    }
};
vector<vector<int>>> tree;          // 最终的 MST
vector<int> parent(n + 1, 0);    // 每个节点的父节点

ll prim(vector<vector<pair<int, int>>>& mp, int start, int n) {
    ll sum = 0, cnt = 0;
    priority_queue<state> pq;
    vector<bool> vis(n + 1, 0);
    vector<ll> dis(n + 1, INF);
    pq.push({start, 0});
    dis[start] = 0;
    while (!pq.empty()) {
        auto [u, w] = pq.top(); pq.pop();
        if (vis[u]) continue;
        if (dis[u] != w) continue;
        vis[u] = 1;
        sum += w;
        cnt++;
        if (parent[u] != 0) {    // 保存 MST
            tree[u].push_back(parent[u]);
            tree[parent[u]].push_back(u);
        }
        for (auto& [v, w1]: mp[u]) {
            if (!vis[v] && w1 < dis[v]) {
                dis[v] = w1;
                parent[v] = u;
                pq.push({v, w1});
            }
        }
    }
    return cnt == n ? sum : -1;
}
```

最小生成树（Kruskal 算法） $O(m \log m)$

// 并查集模版

// 结构体

```
struct state {
    int u, v; ll w;
    bool operator<(const state& other) const {
        return w < other.w;
    }
};
```

// Kruskal 方法

vector<state> edge; // 题目给与的图

vector<vector<int>> tree; // 最终的 MST

// [步骤]保存边

sort(range(edge));

ll sum = 0, cnt = 0;

UnionFind b(n);

for (auto& [u, v, w]: edge) {

if (b.merge(u, v)) {

sum += w;

cnt++;

tree[u].push_back(v);

tree[v].push_back(u);

if (cnt == n - 1) break;

}

}

最近公共祖先 LCA

> 预处理 $O(n\log n)$, 单查 $O(\log n)$

```
vector<vector<int>> mp, f;    // 图和起始数组
vector<int> dep;             // 点深度
const int LOGN = 20;        // 根据点最大值设置
```

// 基本函数

```
int getLCA(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    // 先递归两个节点到同一高度
    for (int i = LOGN - 1; i >= 0; i--)
        if (dep[f[u][i]] >= dep[v]) u = f[u][i];
    if (u == v) return u;
    // 然后齐头并进
    for (int i = LOGN - 1; i >= 0; i--)
        if (f[u][i] != f[v][i])
            u = f[u][i], v = f[v][i];
    return f[u][0];
}
```

// DFS 初始化

```
void dfs(int u, int fa, int d) {
    dep[u] = d;
    f[u][0] = fa;
    path.push_back(u);
    for (int i = 1; i < LOGN; i++)
        f[u][i] = f[f[u][i - 1]][i - 1];
    for (int& v: mp[u]) {
        if (v != fa) dfs(v, u, d + 1);
    }
}
```

// BFS 初始化, 防止树链过长导致爆栈

```
dep[s] = 1;
f[s][0] = s;
queue<int> q;
q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    path.push_back(u);
    for (int i = 1; i < LOGN; i++)
        f[u][i] = f[f[u][i - 1]][i - 1];
    for (int& v: mp[u]) {
        if (v != f[u][0]) {
            dep[v] = dep[u] + 1;
            f[v][0] = u;
            q.push(v);
        }
    }
}
```

树上差分 $O(n + m)$

// 需要在 LCA 初始化中保存访问树节点的序列，只需保证父节点比子节点先访问即可

```
vector<int> diff(n + 1);
while (k--) {
    int u, v; cin >> u >> v;
    diff[u]++; // 两节点本身加
    diff[v]++;
    int l = getLCA(u, v);
    diff[l]--; // 两节点共同父节点和该父节点的父节点减
    if (f[l][0] != root) diff[f[l][0]]--;
}
int ans = 0;
for (auto it = path.rbegin(); it != path.rend(); it++) {
    int& u = *it;
    ans = max(ans, diff[u]);
    // 子节点加回父节点，前缀和还原差分数组
    if (f[u][0] != root) diff[f[u][0]] += diff[u];
}
```

LCA 扩展

1. 路径节点数: $count(u, v) = dep(u) + dep(v) - 2 * dep(LCA(u, v)) + 1$
2. 两点距离: $dis(u, v) = dep(u) + dep(v) - 2 * dep(LCA(u, v))$

Dijkstra 单源最短路径【仅能在非负权图中使用】

> 仅非负权可用，标准 $O((n+m)\log n)$ ，分层图 $O(k(n+m) \cdot \log(kn))$

> 扩展: 0-1BFS，双端队列，仅边权为 0 和 1 使用，权 0 入队头，权 1 入队尾

```
const ll INF = 4e18;
```

```
struct state {  
    int node;    // 当前节点  
    ll dis;      // 前往节点的距离[权值]  
    state(int n, ll d): node(n), dis(d) {}  
    // 小顶堆[确保升序第一个最小]  
    bool operator<(const state& other) const {  
        return dis > other.dis;  
    }  
};
```

```
vector<ll> Dijkstra(int start) {  
    vector<ll> dis(n + 1, INF);  
    vector<int> parent(n + 1, -1);  
    priority_queue<state> q;  
    q.push(state(start, 0LL));  
    dis[start] = 0;  
    while (!q.empty()) {  
        auto [u, w] = q.top(); q.pop();  
        if (dis[u] != w) continue;  
        for (auto& [v, w2]: mp[u]) {  
            if (w + w2 < dis[v]) {  
                dis[v] = w + w2;  
                parent[v] = u;  
                q.push({v, w + w2});  
            }  
        }  
    }  
    return dis;  
}
```

秦九韶算法

对于多项式 $a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$

可将其分解为 $(\dots((a_n x + a_{n-1})x + a_{n-2})x + \dots + a_1)x + a_0$

证明:

$$\begin{aligned} & 5x^4 + 6x^3 + 7x^2 + 8x \\ &= (5x^3 + 6x^2 + 7x + 8)x \\ &= ((5x^2 + 6x + 7)x + 8)x \\ &= (((5x + 6)x + 7)x + 8)x \end{aligned}$$

逆元

在模 m 的条件下, 如果存在一个整数 x , 使得 $a \cdot x \equiv 1 \pmod{m}$

那么这个 x 就是 a 在模 m 下的逆元, 记作 $a^{-1} \pmod{m}$, 即 $x = a^{-1}$

同时 a 和 m 互质 (即 a 和 m 的最大公因数是 1), 互质的情况下一定能找到 x

逆元的存在就是求得模 m 的条件下, a^{-1} 的正确值

如果直接计算 a^{-1} 会在取整后得到 0, 与模运算下的除法结果不正确, 即直接算分数除法在整数模下不合法

组合数公式 $C_n^k = \frac{n!}{k! \cdot (n-k)!}$

费马小定理

如果 p 是质数, 且 a 与 p 互质 ($\gcd(a, p) = 1$), 则 $a^{p-1} \equiv 1 \pmod{p}$

那么 $a^{p-1} \equiv a \cdot a^{p-2} \equiv 1 \pmod{p}$, 即 $a^{-1} = a^{p-2} \pmod{p}$

所以对于模 p 情况下求 a^{-1} , 等价于直接计算 a^{p-2}

// 步骤 1: 预处理阶乘

```
const int MAX = 1000000;
```

```
const int MOD = 1e9 + 7; // 若题目要求需更换
```

```
vector<ll> fact(MAX + 1);
```

```
fact[0] = 1;
```

```
for (int i = 1; i <= MAX; i++) fact[i] = fact[i - 1] * i % MOD;
```

```
/* 快速幂模版 fast_pow */
```

// 步骤 2: 组合数公式

```
ll C(int n, int k) {
```

```
    if (k < 0 || k > n) return 0;
```

```
    return fact[n] * fast_pow(fact[k], MOD-2) % MOD * fast_pow(fact[n-k], MOD-2) % MOD;
```

```
}
```

扩展欧几里得定理

对于任意整数 A 和 B , 一定存在整数解 (x, y) 使得 $Ax + By = \gcd(A, B)$

求解模 p 情况下的 a^{-1} , 即求解整数 x 使得模 p 情况下 $x = a^{-1}$, 且 $a \cdot x \equiv 1 \pmod{p}$

此时根据模的定义, 还存在整数 y 使得 $ax - py = 1$

又因对于整数 a 和 p 存在 $ax + py = \gcd(a, p)$

如果 $\gcd(a, p) = 1$, 那么 $ax + py = 1$, 移项 $ax = 1 - py$

两边同取余 p 可得 $ax \equiv (1 - py) \pmod{p} \equiv (1 \pmod{p} - py \pmod{p}) \equiv 1 \pmod{p}$

此时求解扩展欧几里得中的整数 x , 即可求得模 p 情况下的 a^{-1}

// 求出 $ax + by = \gcd(a, b)$

```
ll exgcd(ll a, ll b, ll& x, ll& y) {
```

```
    if (b == 0) { x = 1, y = 0; return a; }
```

```
    ll g = exgcd(b, a % b, y, x);
```

```
    y -= a / b * x; return g;
```

```
}
```

// 求逆元(即得到求余情况下的 a^{-1})

```
ll inv(ll a, ll mod) {
```

```
    ll x, y;
```

```
    ll g = exgcd(a, mod, x, y);
```

```
    if (g != 1) return -1;
```

```
    return (x % mod + mod) % mod;
```

```
} /* 后面通过公式求解组合数即可 */
```

线段树 懒标记完整版

> 建树 $O(n)$, 单次操作 $O(\log n)$

vector<ll> num, tree, lazy; // tree 和 lazy 要开 4 倍

// 线段树本体

```
void push_up(int p) { tree[p] = tree[p << 1] + tree[p << 1 | 1]; }
```

// 懒标记处理

```
void apply(int p, int l, int r, ll k) {  
    tree[p] += (r - l + 1) * k;  
    lazy[p] += k;  
}
```

// 乘法懒标记需要初始化为 1

// 加乘都有时, 需要按 $(a+b)c=ac+bc$ 先处理乘法懒标记再处理加分懒标记

// 处理乘法懒标记时需要同时处理加分懒标记(给加分懒标记乘上对应值)

// 含有覆盖操作时, 需优先处理覆盖懒标记, 覆盖标记需要使用额外数组, 且需要同时清空其他运算懒标记

// 懒标记下传

```
void push_down(int p, int l, int r) {  
    if (l == r || !lazy[p]) return;  
    int mid = (l + r) >> 1;  
    apply(p << 1, l, mid, lazy[p]);  
    apply(p << 1 | 1, mid + 1, r, lazy[p]);  
    lazy[p] = 0;  
}
```

// 初始化

```
void build(int p, int l, int r) {  
    if (l == r) {  
        tree[p] = num[l];  
        return;  
    }  
    int mid = (l + r) >> 1;  
    build(p << 1, l, mid);  
    build(p << 1 | 1, mid + 1, r);  
    push_up(p);  
}
```

// 区间更新

```
void update(int p, int l, int r, int ql, int qr, ll k) {  
    if (ql <= l && r <= qr) {  
        apply(p, l, r, k);  
        return;  
    }  
    push_down(p, l, r);  
    int mid = (l + r) >> 1;  
    if (ql <= mid) update(p << 1, l, mid, ql, qr, k);  
    if (mid < qr) update(p << 1 | 1, mid + 1, r, ql, qr, k);  
    push_up(p);  
}
```

// 区间查询

```
ll query(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tree[p];
    push_down(p, l, r);
    int mid = (l + r) >> 1;
    ll ans = 0;
    if (ql <= mid) ans += query(p << 1, l, mid, ql, qr);
    if (mid < qr) ans += query(p << 1 | 1, mid + 1, r, ql, qr);
    return ans;
}
```


树状数组（动态前缀和） $O(\log n)$

```
struct bit {
    int n;
    vector<ll> tree;
    bit(int n1): n(n1), tree(n + 2, 0) {}
    void add(int i, ll delta) {
        if (i <= 0) return;
        for (; i <= n; i += i & -i)
            tree[i] += delta;
    }
    ll query(int i) {
        ll sum = 0;
        for (; i > 0; i -= i & -i)
            sum += tree[i];
        return sum;
    }
};
```

对拍器

> 需准备好`a.exe` (暴力)、`b.exe` (优化)、`in.txt` (输入)、`out.txt` (输出)

```
mt19937 rng(time(0));
ll rnd(ll l, ll r) {    // 生成随机数
    return uniform_int_distribution<ll>(l, r)(rng);
}

void gen() {
    ofstream fout("in.txt");
    // =====
    // 根据实际情况输入修改
    int n = rnd(1, 10);
    fout << n << "\n";
    for (int i = 0; i < n; i++) {
        fout << rnd(1, 1e18) << " " << rnd(1, 1e18) << " " << rnd(1, 1e18) << "\n";
    }
    // =====
    fout.close();
}

int main(){
    for(int t = 1; t <= 100000; t++){
        gen(); // 生成数据
        if(system("a.exe < in.txt > a.txt") != 0) { // 跑暴力
            cout << "\na.exe crashed!" << endl;
            break;
        }
        auto start = chrono::high_resolution_clock::now();
        if(system("b.exe < in.txt > b.txt") != 0) { // 跑正解
            cout << "\nb.exe crashed!" << endl;
            break;
        }
        auto end = chrono::high_resolution_clock::now();
        chrono::duration<double> diff = end - start;
        if(diff.count() > 1.0 || system("fc a.txt b.txt > nul") != 0){ // 输出错误(计时超过 1 秒或答案不对)
            cout << "Wrong Answer!" << endl;
            cout << "Test Case: " << t << endl;
            cout << "--- Input ---" << endl;
            system("type in.txt");
            cout << "\n--- Expected (a.txt) ---" << endl;
            system("type a.txt");
            cout << "\n--- Actual (b.txt) ---" << endl;
            system("type b.txt");
            break;
        }
        cout << "Test " << t << " OK" << endl;
    }
}
```

对拍器 py 跨 c++

> 需存在`a.exe`(由`a.cpp`编译而来, 优化或暴力)、`b.py`(优化或暴力)、`out\_a.txt`(`a.cpp`输出)、`out\_b.txt`(`b.py`输出)、`in.txt`(输入)

```
import subprocess
```

```
import random
```

```
import os
```

```
CMD_A = ["/a.exe"]          # C++ 程序 (确保已编译)
```

```
CMD_B = ["pypy3", "b.py"]   # Python 程序
```

```
FILE_IN = "in.txt"
```

```
FILE_OUT_A = "out_a.txt"
```

```
FILE_OUT_B = "out_b.txt"
```

```
def generate_data():
```

```
    # 数据生成区
```

```
    q = random.randint(1, 10)
```

```
    with open(FILE_IN, "w") as f:
```

```
        f.write(f'{q}\n')
```

```
    for _ in range(q):
```

```
        l = random.randint(1, 2)
```

```
        r = random.randint(1, 10)
```

```
        f.write(f'{l} {r}\n')
```

```
def run_program(cmd, input_file, output_file):
```

```
    with open(input_file, 'r') as fin, open(output_file, 'w') as fout:
```

```
        subprocess.run(cmd, stdin=fin, stdout=fout, stderr=subprocess.DEVNULL)
```

```
def main():
```

```
    if not os.path.exists("a.exe"):
```

```
        print("错误: 找不到 a.exe, 请先编译 a.cpp")
```

```
    return
```

```
    cnt = 0
```

```
    while True:
```

```
        cnt += 1
```

```
        print(f"Testing Case #{cnt}...", end=" ")
```

```
        generate_data()
```

```
        run_program(CMD_A, FILE_IN, FILE_OUT_A)
```

```
        run_program(CMD_B, FILE_IN, FILE_OUT_B)
```

```
        with open(FILE_OUT_A, 'r') as f1, open(FILE_OUT_B, 'r') as f2:
```

```
            res_a = f1.read().strip()
```

```
            res_b = f2.read().strip()
```

```
        if res_a == res_b:
```

```
            print(" AC")
```

```
        else:
```

```
            print(" WA!")
```

```
            print("\n" + "="*30)
```

```
            with open(FILE_IN, 'r') as f:
```

```
                print(f"Input:\n {f.read()}")
```

```
            print(f"Output A (a.cpp): {res_a}")
```

```
            print(f"Output B (b.py): {res_b}")
```

```
            print("="*30)
```

```
            break
```

```
if __name__ == "__main__": main()
```

埃氏筛 找出小于或等于 n 的素数

```
vector<bool> prime(n+1, true);
prime[0] = prime[1] = false;
for (int i = 2; i*i <= n; i++)
    if (prime[i])
        for (ll p = 1LL * i * i; p <= n; p += i)
            prime[p] = false;
```

快速幂 计算 a^b , pow(a, b)的优化版

```
const int MOD = 1e9 + 7;
long long fast_pow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}
```

数学知识

1. 等差数列的首项为 a_1 ，公差为 n ，则其通项公式为 $a_n = a_1 + (n - 1) * d$,

求和公式为 $S_n = n * a_1 + \frac{n * (n-1)}{2} * d$ 。

2. 等比数列的首项为 a_1 ，公比为 q ，则其通项公式为 $a_n = a_1 * q^{n-1}$,

求和公式为 $S_n = a_1 * \frac{1-r^n}{1-r}$ 。

3. 素数等于质数，要求均为因子只有 1 和他本身。

4. 如果 t 为 MEX（集合中没有出现的最小自然数），则集合中必有 $\{0, \dots, t-1\}$ 的完整序列并且不存在 t。

5. 设三点为 A (x_1, y_1), B (x_2, y_2), C (x_3, y_3)。

如果三点共线，必满足 $(x_1 - x_2) * (y_3 - y_1) - (y_2 - y_1) * (x_3 - x_1) = 0$ 。

三点所在三角形面积： $S = \frac{1}{2} * abs((x_2 - x_1) * (y_3 - y_1) - (y_2 - y_1) * (x_3 - x_1))$ 。

6. 拉格朗日四平方和定理：任何一个正整数，都可以写成至多 4 个平方数的和。

7. 勒让德三平方和定理：当 $n \neq 4^a(8b + 7)$ ，其中 $a, b \in \mathbb{Z}$ ，n就可以表示为三个平方数之和。

8. 裴蜀定理：所有形如 $ax + by$ 的整数，恰好是 $\gcd(x, y)$ 的倍数